🗺️ TRIE · PREFIX TREE

FAANG Masterclass · Java Edition

💡 WHAT IS A TRIE?

A **prefix tree** (digital tree) stores characters as nodes. Each path from root to node represents a prefix.

> 🔑 **AHA:** Strings that share a prefix (e.g. "cat", "cater", "cats") reuse the same branch `c → a → t`. Saves space & enables instant prefix search!

📊 COMPLEXITY

Operation	Time	Space
Insert (Length L)	**O(L)**	O(L·N)
Search (Length L)	**O(L)**	O(1)
StartsWith (Length L)	**O(L)**	O(1)

✅ **Why O(L)?** Array indexing `children[c-'a']` is O(1), so we just walk L characters.

🔥 TRIE vs HASHMAP

`HashMap` is O(1) for **exact** lookups, but CANNOT search by prefix. Trie gives **O(L) prefix search** independent of dictionary size N.

> 🧠 **AHA:** "autocomplete", "search suggestions", "prefix match" → **Trie**

1 TRIE NODE STRUCTURE

```
public class TrieNode {
    TrieNode[] children = new TrieNode[26];  // lowercase English
    boolean isWord = false;
    // optional: String word;  // for Word Search II
    // optional: int val;      // for MapSum
}
```

📦 IMPLEMENT TRIE

LC 208 · Insert · Search · StartsWith

🔧 STANDARD TRIE — LC 208

```
class Trie {
    private final TrieNode root = new TrieNode();

    public void insert(String word) {
        TrieNode curr = root;
        for (char c : word.toCharArray()) {
            int idx = c - 'a';
            if (curr.children[idx] == null)
                curr.children[idx] = new TrieNode();
            curr = curr.children[idx];
        }
        curr.isWord = true;
    }

    public boolean search(String word) {
        TrieNode node = find(word);
        return node != null && node.isWord;
    }

    public boolean startsWith(String prefix) {
        return find(prefix) != null;
    }

    private TrieNode find(String s) {
        TrieNode curr = root;
        for (char c : s.toCharArray()) {
            int idx = c - 'a';
            if (curr.children[idx] == null) return null;
            curr = curr.children[idx];
        }
        return curr;
    }
}
```

🔍 DRY RUN — INSERT "app"

Char	Idx	Child exists?	Action
'a'	0	❌	create → move
'p'	15	❌	create → move
'p'	15	❌	create → move
END	—	—	`curr.isWord = true` ✅

Search "app" → root → a → p → p → `isWord == true` ✅
Search "appl" → root → a → p → p → l → `null` ❌

> 🧠 **AHA:** Search and StartsWith share the same `find()` method; only difference is checking `isWord`.

DECISION TREE

When to use which Trie variant?

PREFIX PROBLEMS → VARIANT

Exact prefix search / autocomplete	→	Standard Trie (children[26])
Wildcard '.' search	→	Trie + DFS (LC 211)
Shortest root replacement	→	Prefix Match (LC 648)
Autocomplete top 3	→	Store lists at nodes (LC 1268)
Multi-word grid search	→	Trie + Grid DFS (LC 212)
Prefix sum (MapSum)	→	Trie + DFS subtree / delta
Suffix encoding	→	Reverse Trie (LC 820)
Maximum XOR	→	Bit Trie (2 children) (LC 421)

TRIGGER WORDS → VARIANT

Trigger Word	Variant	Key Insight
"autocomplete", "suggestions"	Standard / Top-K	Store lists at nodes
"wildcard dot"	Trie + DFS	On '.' loop all 26 children
"word boggle", "grid words"	Trie + Grid DFS	Prune invalid paths
"max XOR"	Bit Trie (2 children)	Greedy opposite bit

WILDCARD & SHORTEST PREFIX

LC 211 · Add & Search Words · LC 648 · Replace Words

LC 211 · Add & Search Words

Input: addWord("bad"), search(".ad") → **Output:** true

Input: search("b..") → **Output:** true

Pattern: Wildcard DFS — on '.' loop all 26 children.

AHA: When we see `.`, we must **DFS over all 26 children**. If **any** branch returns true → true.

```
public boolean search(String word) {
    return searchInNode(word, 0, root);
}

private boolean searchInNode(String word, int idx, TrieNode node) {
    if (node == null) return false;
    if (idx == word.length()) return node.isWord;

    char c = word.charAt(idx);
    if (c == '.') {
        for (TrieNode child : node.children) {
            if (child != null && searchInNode(word, idx + 1, child))
                return true;
        }
        return false;
    } else {
        return searchInNode(word, idx + 1, node.children[c - 'a']);
    }
}
```

COMPLEXITY: O(26^L) worst-case for wildcard, but Trie prunes branches.

LC 648 · Replace Words

Input: dict=["cat","bat"], sentence="cattle battery" → **Output:** "cat bat"

Pattern: shortest root prefix — stop when `isWord == true`.

AHA: As we traverse the Trie character by character, **stop immediately** when we hit `curr.isWord == true` — that's the shortest root! Don't go deeper.

```
public String replaceWords(List<String> dict, String sentence) {
    Trie trie = new Trie();
    for (String root : dict) trie.insert(root);

    String[] words = sentence.split(" ");
    StringBuilder sb = new StringBuilder();
    for (String w : words) {
        if (sb.length() > 0) sb.append(" ");
        sb.append(trie.findRoot(w));
    }
    return sb.toString();
}

// Inside Trie class:
public String findRoot(String word) {
    TrieNode curr = root;
    StringBuilder prefix = new StringBuilder();
    for (char c : word.toCharArray()) {
        int idx = c - 'a';
        if (curr.children[idx] == null || curr.isWord) break;
        prefix.append(c);
        curr = curr.children[idx];
    }
    return curr.isWord ? prefix.toString() : word;
}
```

🗺️ GRID DFS & MAP SUM

LC 212 · Word Search II · LC 677 · Map Sum Pairs

🗺️ LC 212 · Word Search II

Input: board=[["o","a"],["t","h"]], words=["oath"] → **Output:** ["oath"]

🔴 **WHY TRIE BEATS PLAIN DFS:** Searching M words with plain DFS → run grid DFS M times → O(M · 4^L) → TLE. Insert all words into a **single Trie**. Then one grid DFS searches **all words simultaneously**, pruning invalid branches instantly.

🧠 **AHA:** Store the full `word` at the leaf node. When we find it, add to result and set `node.word = null` to avoid duplicates!

```
// Direction vectors for 4-neighbor traversal
private static final int[][] DIRS = {{0,1},{0,-1},{1,0},{-1,0}};

public List<String> findWords(char[][] board, String[] words) {
    // 1. Build Trie from all words
    TrieNode root = new TrieNode();
    for (String w : words) {
        TrieNode curr = root;
        for (char c : w.toCharArray()) {
            int idx = c - 'a';
            if (curr.children[idx] == null)
                curr.children[idx] = new TrieNode();
            curr = curr.children[idx];
        }
        curr.word = w;  // Store full word at leaf
    }

    // 2. DFS from each cell
    List<String> res = new ArrayList<>();
    for (int r = 0; r < board.length; r++) {
        for (int c = 0; c < board[0].length; c++) {
            dfs(board, r, c, root, res);
        }
    }
    return res;
}

private void dfs(char[][] board, int r, int c,
                 TrieNode node, List<String> res) {
    char ch = board[r][c];
    // Prune: visited or no matching child
    if (ch == '#' || node.children[ch - 'a'] == null) return;

    // Move to child
    node = node.children[ch - 'a'];

    // Found a word — add and clear to avoid duplicates
    if (node.word != null) {
        res.add(node.word);
        node.word = null;  // ⭐ deduplicate
    }

    // Mark visited
    board[r][c] = '#';

    // Explore 4 directions
    for (int[] d : DIRS) {
        int nr = r + d[0], nc = c + d[1];
        if (nr >= 0 && nr < board.length &&
            nc >= 0 && nc < board[0].length) {
            dfs(board, nr, nc, node, res);
        }
    }

    // Backtrack — restore character
    board[r][c] = ch;
}
```

Why restore? After exploring all directions, we **restore** the original character so other paths can reuse this cell. Standard **backtracking**.

🧮 LC 677 · Map Sum Pairs

Input: insert("apple",3), sum("ap") → **Output:** 3

🧠 **AHA:** Reach the prefix node, then **DFS the entire subtree** and sum all `isWord` values.

```
// Modified TrieNode with val
class TrieNode {
    TrieNode[] children = new TrieNode[26];
    boolean isWord = false;
    int val = 0;  // value if isWord
}

public int sum(String prefix) {
    TrieNode node = find(prefix);
    if (node == null) return 0;
    return dfsSum(node);
}

private int dfsSum(TrieNode node) {
    int total = node.isWord ? node.val : 0;
    for (TrieNode child : node.children) {
        if (child != null) total += dfsSum(child);
    }
    return total;
}
```

⚡ **OPTIMIZED — DELTA:** Store `node.sum` = sum of all values in subtree. When updating a key, compute **delta** = newVal - oldVal, and add delta to **every node** along the path → O(L) per insert!

```
// insert("apple", 3) → nodes: a,p,p,l,e each += 3
// insert("apple", 6) → delta = 3 → add 3 to path
// sum("app") → just return node.sum at "app"
```

LONGEST WORD & REVERSE TRIE

LC 720 · Longest Word in Dictionary · LC 820 · Short Encoding

LC 720 · Longest Word

Input: words=["a","banana","app","appl","ap","apply","apple"] → **Output:** "apple"

🧠 **AHA:** We need to traverse the Trie, but only move to a child **if** `child.isWord == true`. Because every prefix must be a valid word.

```
public String longestWord(String[] words) {
    Trie trie = new Trie();
    for (String w : words) trie.insert(w);

    String[] best = {""};
    dfs(trie.root, new StringBuilder(), best);
    return best[0];
}

private void dfs(TrieNode node, StringBuilder sb, String[] best) {
    for (int i = 0; i < 26; i++) {
        TrieNode child = node.children[i];
        if (child != null && child.isWord) {
            sb.append((char)('a' + i));
            if (sb.length() > best[0].length()) {
                best[0] = sb.toString();
            }
            dfs(child, sb, best);
            sb.deleteCharAt(sb.length() - 1);
        }
    }
}
```

🔍 **DRY RUN:** root → a (isWord) → p (isWord) → p (isWord) → l (isWord) → e (isWord) → "apple" ✅
❌ "banana": root → b → but "b" is not a word → cannot start.

LC 820 · Short Encoding

Input: words=["time","me","emit"] → **Output:** 10 (time#emit#)

🧠 **AHA:** If word B is a **suffix** of word A, we don't need to add B separately. We can **reverse** each word → suffix becomes **prefix** → use a standard Trie!

```
public int minimumLengthEncoding(String[] words) {
    TrieNode root = new TrieNode();
    // Insert reversed words
    for (String w : words) {
        TrieNode curr = root;
        for (int i = w.length() - 1; i >= 0; i--) {
            char c = w.charAt(i);
            int idx = c - 'a';
            if (curr.children[idx] == null)
                curr.children[idx] = new TrieNode();
            curr = curr.children[idx];
        }
        curr.isWord = true;
    }
    return dfs(root, 0);
}

private int dfs(TrieNode node, int depth) {
    boolean isLeaf = true;
    int sum = 0;
    for (TrieNode child : node.children) {
        if (child != null) {
            isLeaf = false;
            sum += dfs(child, depth + 1);
        }
    }
    if (isLeaf) return depth + 1; // +1 for '#'
    return sum;
}
```

🔍 **EXAMPLE:** words=["time","me","emit"] → reverse: "emit", "em", "time" → "me" is NOT leaf (it's part of "time") → only leaf nodes count!

BITWISE TRIE & CHEAT SHEET

LC 421 · Maximum XOR · Golden Rules

LC 421 · Maximum XOR of Two Numbers

Input: nums=[3,10,5,25,2,8] → **Output:** 28 (5 ^ 25)

🧠 **AHA:** To maximize XOR, at each bit **greedily try the OPPOSITE bit** (`1 ^ bit`). If it exists, take it; otherwise take the same bit.

Bit Node & Insert:

```
class BitNode {
    BitNode[] child = new BitNode[2];
}

void insert(int num) {
    BitNode curr = root;
    for (int i = 30; i >= 0; i--) {
        int bit = (num >> i) & 1;
        if (curr.child[bit] == null)
            curr.child[bit] = new BitNode();
        curr = curr.child[bit];
    }
}
```

Max XOR Search:

```
int maxXor = 0;
for (int num : nums) {
    BitNode curr = root;
    int xor = 0;
    for (int i = 30; i >= 0; i--) {
        int bit = (num >> i) & 1;
        int opp = 1 ^ bit;
        if (curr.child[opp] != null) {
            xor |= (1 << i);
            curr = curr.child[opp];
        } else {
            curr = curr.child[bit];
        }
    }
    maxXor = Math.max(maxXor, xor);
}
return maxXor;
```

✅ MASTERY CHECKLIST

- ☐ Implement Trie from scratch (3 min)
- ☐ Wildcard '.' search with DFS
- ☐ Shortest prefix replacement (LC 648)
- ☐ Search Suggestions (Top 3)
- ☐ Word Search II (Trie + Grid DFS)
- ☐ MapSum (DFS or delta)
- ☐ Longest Word (child.isWord check)
- ☐ Reverse Trie (suffix encoding)
- ☐ Bit Trie (Max XOR)

🎯 INTERVIEW FOLLOW-UPS

Question	Your Answer
Trie vs HashMap?	HashMap O(1) exact, but no prefix search. Trie O(L) prefix.
Array vs HashMap children?	Array faster for 26 chars. HashMap better for Unicode.
How to handle duplicates in Word Search II?	Set `node.word = null` after adding.
Why reverse for suffix encoding?	Suffix becomes prefix → standard Trie!

💡 **GOLDEN RULES:** `'.'` → loop all 26 children (DFS) Stop at `isWord` for shortest prefix Store `word` at leaf for Word Search II Reverse suffix → prefix → standard Trie

Bit Trie: 2 children, greedy opposite bit Longest Word: only move to `child.isWord`